Django Templates — Complete Notes

1. What Are Django Templates?

Django Templates are used to generate dynamic HTML pages by combining static HTML with dynamic data.

A template contains:

- Static content, such as HTML elements and text.
- Variables to display dynamic data.
- Tags to implement logic, such as loops and conditions.
- Filters to modify displayed values.
- Comments that are not included in the rendered output.

Example

views.py

```
from django.shortcuts import render

def home(request):
    context = {
        "first_name": "John",
        "last_name": "Doe",
    }
    return render(request, "home.html", context)
```

templates/home.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Home</title>
</head>
<body>
    <h1>Welcome {{ first_name }} {{ last_name }}</h1>
</body>
</html>
```

Output

```
Welcome John Doe
```

Advantages of Django Templates

- Generate dynamic HTML pages.
- Separate presentation from business logic.
- Reuse common layouts.
- Display data from databases.
- Support loops, conditions, filters, and template inheritance.
- Reduce repeated HTML code.

---

2. Django Template Language (DTL)

The Django Template Language (DTL) is Django's built-in template system.

Django supports multiple template engines, including:

1. Django Template Language (DTL)
2. Jinja2
3. Custom template engines

DTL is a good default choice, particularly when building reusable Django applications.

Four Main Components of DTL

Component	Syntax	Purpose
Variables	`{{ variable }}`	Display data
Tags	`{% tag %}`	Add template logic
Filters	`{{ variable |filter }}`	Transform values

Comments	`{# comment #}`	Add non-rendered comments

---

3. Template Variables

Variables display values passed to a template through its context.

Syntax

```
{{ variable_name }}
```

Example

views.py

```
def profile(request):
    context = {
        "name": "Omar",
        "age": 25,
    }
    return render(request, "profile.html", context)
```

profile.html

```
<h1>Name: {{ name }}</h1>
<p>Age: {{ age }}</p>
```

Output

```
Name: Omar
Age: 25
```

Dictionary Lookup

```
{{ user.name }}
{{ user.email }}
```

Object Attribute Lookup

```
{{ product.name }}
{{ product.price }}
```

List Index Lookup

```
{{ students.0 }}
{{ students.1 }}
```

DTL uses dot notation for dictionary keys, object attributes, and list indexes.

Important: Django templates are not Python code. You cannot use arbitrary Python expressions directly inside `{{ }}`.

---

4. Template Tags

Template tags perform operations during template rendering.

4.1 If Tag

The `if` tag conditionally displays content.

```
{% if user.is_authenticated %}
    <h1>Welcome, {{ user.username }}</h1>
{% else %}
    <h1>Please log in</h1>
{% endif %}
```

4.2 Elif Tag

```
{% if marks >= 80 %}
    <p>Grade: A+</p>
{% elif marks >= 60 %}
    <p>Grade: A</p>
{% else %}
    <p>Grade: B or below</p>
{% endif %}
```

4.3 For Tag

The `for` tag iterates over a sequence.

views.py

```
def students_view(request):
    students = ["Rahim", "Karim", "Omar"]
    return render(
        request,
        "students.html",
```

```
            {"students": students},
        )
```

students.html

```
<ul>
    {% for student in students %}
        <li>{{ student }}</li>
    {% empty %}
        <li>No students found.</li>
    {% endfor %}
</ul>
```

The `{% empty %}` clause displays content when the sequence is empty.

4.4 Forloop Variables

Django provides special variables inside a `for` loop.

Variable	Meaning
`forloop.counter`	Current iteration number, starting at 1
`forloop.counter0`	Current iteration number, starting at 0
`forloop.first`	True on the first iteration

`forloop.last`	True on the last iteration
`forloop.revcounter`	Remaining iterations, starting at 1
`forloop.revcounter0`	Remaining iterations, starting at 0

Example:

```
{% for student in students %}
    <p>{{ forloop.counter }}. {{ student }}</p>
{% endfor %}
```

4.5 URL Tag

The `url` tag generates URLs from named URL patterns.

urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path("profile/", views.profile, name="profile"),
]
```

template.html

```
<a href="{% url 'profile' %}">My Profile</a>
```

For a URL requiring a parameter:

```
path("product/<int:product_id>/", views.product_detail,
name="product_detail")
```

```
<a href="{% url 'product_detail' product.id %}">
    View Product
</a>
```

4.6 Static Tag

The `static` tag generates URLs for static files such as CSS, JavaScript, and images.

At the top of the template:

```
{% load static %}
```

Then use:

```
<link rel="stylesheet" href="{% static 'css/style.css' %}">
<script src="{% static 'js/app.js' %}"></script>
<img src="{% static 'images/logo.png' %}" alt="Logo">
```

4.7 CSRF Token Tag

The `csrf_token` tag adds a CSRF token to a POST form.

```
<form method="post">
    {% csrf_token %}
    <input type="text" name="username">
    <button type="submit">Submit</button>
</form>
```

It helps protect against Cross-Site Request Forgery (CSRF) attacks.

4.8 With Tag

The `with` tag assigns a value to a local template variable.

```
{% with total=products.count %}
    <p>Total products: {{ total }}</p>
{% endwith %}
```

Use this to avoid repeating an expression within a template section.

4.9 Comment Tag

Single-line comment:

```
{# This comment will not appear in the output. #}
```

Multi-line comment:

```
{% comment %}
This is a multi-line comment.
It will not be rendered.
{% endcomment %}
```

---

5. Template Filters

Filters transform variable values before displaying them.

Syntax

```
{{ variable|filter_name }}
```

A filter can accept an argument:

```
{{ variable|filter_name:"argument" }}
```

Commonly Used Filters

Filter	Example	Purpose
`lower`	`{{ name|lower }}`	Convert to lowercase
`upper`	`{{ name|upper }}`	Convert to uppercase
`title`	`{{ name|title }}`	Capitalize word beginnings
`capfirst`	`{{ name|cap`	Capitalize the first

	`first }}`	character
`length`	`{{ items|length }}`	Return the length
`default`	`{{ name|default:"Guest" }}`	Use a fallback for false values
`date`	`{{ created_at|date:"Y-m-d" }}`	Format a date
`time`	`{{ created_at|time:"H:i" }}`	Format a time
`truncatechars`	`{{ text|truncatechars:20 }}`	Truncate to a character limit
`join`	`{{ items|join:", " }}`	Join sequence items

`safe`	`{{ content|safe }}`	Disable HTML escaping for a value
`linebreaks`	`{{ text|linebreaks }}`	Convert line breaks to HTML paragraphs
`wordcount`	`{{ text|wordcount }}`	Count words
`floatformat`	`{{ price|floatformat:2 }}`	Format a number to decimal places

Examples

Lowercase

```
{{ "DJANGO"|lower }}
```

Output:

```
django
```

Uppercase

```
{{ "django"|upper }}
```

Output:

```
DJANGO
```

Default Value

```
{{ username|default:"Guest" }}
```

If `username` is empty or otherwise false, the output is:

```
Guest
```

Date Formatting

```
{{ post.created_at|date:"d M Y" }}
```

Possible output:

```
14 Oct 2026
```

Truncate Text

```
{{ description|truncatechars:20 }}
```

The filter limits the displayed text to approximately 20 characters, including the truncation marker when needed.

Important: Django automatically escapes many variable values in HTML templates. Avoid using `safe` with untrusted user input, because it can introduce cross-site scripting (XSS) vulnerabilities.

---

6. Template Inheritance

Template inheritance allows multiple pages to share a common layout.

It helps avoid repeating the same HTML structure on every page.

6.1 Base Template

templates/base.html

```
<!DOCTYPE html>
<html>
<head>
    <title>{% block title %}My Website{% endblock %}</title>
</head>
<body>
```

```
        <header>
            <h1>My Website</h1>
        </header>

        <nav>
            <a href="/">Home</a>
        </nav>

        <main>
            {% block content %}
            {% endblock %}
        </main>

        <footer>
            <p>Copyright 2026</p>
        </footer>
    </body>
    </html>
```

6.2 Child Template

templates/home.html

```
    {% extends "base.html" %}

    {% block title %}
    Home Page
    {% endblock %}

    {% block content %}
        <h2>Welcome to Django</h2>
        <p>This is the home page.</p>
    {% endblock %}
```

Important Inheritance Tags

- `{% extends "base.html" %}` — inherits a parent template.
- `{% block name %}` — defines a replaceable section.
- `{% endblock %}` — ends a block.

Rule: In a child template, `{% extends %}` should normally be the first template tag.

---

7. Template Configuration

Django template engines are configured through the `TEMPLATES` setting in `settings.py`.

Example Configuration

```
TEMPLATES = [
    {
        "BACKEND": "django.template.backends.django.DjangoTemplates",
        "DIRS": [BASE_DIR / "templates"],
        "APP_DIRS": True,
        "OPTIONS": {
            "context_processors": [
                "django.template.context_processors.request",
                "django.contrib.auth.context_processors.auth",
                "django.contrib.messages.context_processors.messages",
            ],
        },
    },
]
```

Important Settings

1. BACKEND

Specifies the template engine.

```
"BACKEND": "django.template.backends.django.DjangoTemplates"
```

2. DIRS

Specifies directories where Django searches for project-level templates.

```
"DIRS": [BASE_DIR / "templates"]
```

3. APP_DIRS

When set to `True`, Django searches for templates inside the `templates/` directories of installed applications.

```
"APP_DIRS": True
```

4. OPTIONS

Contains additional settings for the selected template backend, such as context processors and autoescaping.

Recommended Folder Structure

```
myproject/
│
├── manage.py
├── myproject/
│   ├── settings.py
│   └── urls.py
│
├── templates/
│   ├── base.html
│   └── home.html
│
└── blog/
    ├── views.py
    ├── urls.py
    └── templates/
        └── blog/
            ├── post_list.html
            └── post_detail.html
```

Using an application-specific subdirectory such as `templates/blog/` helps prevent template name conflicts.

---

8. Template Context

A context contains the data made available to a template.

In common Django usage, the context is a dictionary of values.

Example

```
from django.shortcuts import render

def dashboard(request):
    context = {
        "username": "Omar",
        "age": 25,
        "is_active": True,
        "skills": ["Python", "Django", "SQL"],
    }
```

```
    return render(request, "dashboard.html", context)
```

dashboard.html

```
<h1>{{ username }}</h1>
<p>Age: {{ age }}</p>

{% if is_active %}
    <p>Account is active.</p>
{% endif %}

<ul>
    {% for skill in skills %}
        <li>{{ skill }}</li>
    {% endfor %}
</ul>
```

Context vs. Context Processor

- **Context:** Data passed directly to a template, often from a view.
- **Context processor:** A function that receives a request and returns a dictionary of data to add to the rendering context.

Context processors are useful for data needed by many templates, such as authentication information or shared site settings.

Example Custom Context Processor

myapp/context_processors.py

```
def site_info(request):
    return {
        "site_name": "My Website",
    }
```

Register it in `settings.py`:

```
"OPTIONS": {
    "context_processors": [
        "django.template.context_processors.request",
        "django.contrib.auth.context_processors.auth",
        "django.contrib.messages.context_processors.messages",
        "myapp.context_processors.site_info",
```

```
        ],
    },
```

Now templates rendered with a request can access:

```
<h1>{{ site_name }}</h1>
```

---

9. Loading and Rendering Templates

Django provides template-loading functions in `django.template.loader`.

9.1 get_template()

Loads a template and returns a template object.

```
from django.template.loader import get_template

template = get_template("home.html")
```

To render it:

```
html = template.render({"name": "Omar"})
```

When a template cannot be found, Django raises `TemplateDoesNotExist`. Invalid template syntax can raise `TemplateSyntaxError`.

9.2 select_template()

Accepts a list of template names and returns the first template that exists.

```
from django.template.loader import select_template

template = select_template([
    "blog/custom_detail.html",
    "blog/post_detail.html",
])
```

This is useful when a specific page can have a custom template with a default fallback.

9.3 render_to_string()

Loads and renders a template, returning the resulting string.

```
from django.template.loader import render_to_string

html = render_to_string(
    "home.html",
```

```
        {"name": "Omar"},
    )
```

This is useful when you need rendered HTML as a string instead of an HTTP response.

9.4 render()

The `render()` shortcut is the most common way to render a template from a Django view.

```
from django.shortcuts import render

def home(request):
    return render(
        request,
        "home.html",
        {"name": "Omar"},
    )
```

It combines template rendering with an HTTP response.

Comparison

Function	Purpose	Return value
`get_template()`	Load a template	Template object
`select_template()`	Load the first existing template from a list	Template object
`render_to_string()`	Load and render a template	String

`render()`	Render a template for a view response	`HttpResponse`

---

10. Template Loaders

Template loaders locate template files and load them into Django's template system.

With the usual configuration:

```
"APP_DIRS": True
```

Django can find templates in installed applications' `templates/` directories.

Loaders also support searching configured directories and other sources.

You generally do not need to configure loaders manually for a standard Django project.

---

11. Django Template Engines

Django supports different template engines.

11.1 Django Template Language (DTL)

Backend:

```
"BACKEND": "django.template.backends.django.DjangoTemplates"
```

Features:

- Built-in Django template tags and filters.
- Template inheritance.
- Automatic HTML escaping by default.
- Integration with Django applications.

Example:

```
{% if user.is_authenticated %}
    <p>Hello, {{ user.username }}</p>
{% endif %}
```

11.2 Jinja2

Jinja2 is an alternative template engine.

Install it:

```
python -m pip install Jinja2
```

Configure it in `settings.py`:

```
TEMPLATES = [
    {
        "BACKEND": "django.template.backends.jinja2.Jinja2",
        "DIRS": [BASE_DIR / "jinja2_templates"],
        "APP_DIRS": True,
        "OPTIONS": {
            "environment": "myproject.jinja2.environment",
        },
    },
]
```

Jinja2 uses similar but different syntax and provides its own filters, tests, and features.

Example Jinja2 template:

```
{% if user %}
    <h1>Hello, {{ user.username }}</h1>
{% endif %}
```

Django-specific template tags and filters are not automatically available in Jinja2. You must configure equivalent functions or filters when needed.

DTL vs. Jinja2

Feature	DTL	Jinja2
Built into Django	Yes	No, requires installation
Django template tags	Yes	No, not automatically

Template inheritance	Yes	Yes
Filters	Yes	Yes, with different details
Calling functions with arguments	Restricted	Supported
Best default for standard Django templates	Usually	When its features or ecosystem are preferred

---

12. Template Engine API Components

Django exposes several template-related APIs.

Engine

`django.template.Engine` represents an instance of the Django Template Language engine.

It is useful when working with DTL outside the usual project configuration.

Template

A compiled template object represents a template that can be rendered with context data.

Context

`django.template.Context` stores data used during rendering.

RequestContext

`RequestContext` associates rendering with an HTTP request and runs configured context processors.

Template Backend

A backend adapts a template engine to Django's common template API. Django's built-in backends include `DjangoTemplates` and `Jinja2`.

For most applications, you can use `render()` without interacting directly with these lower-level APIs.

---

13. Template Partials in Django 6.0

Django 6.0 introduced support for defining and loading template partials with the Django Template Language backend.

A partial is a reusable fragment of a template.

Example: Define a Partial

```
{% partialdef user_card %}
    <div class="user-card">
        <h2>{{ user.username }}</h2>
        <p>{{ user.email }}</p>
    </div>
{% endpartialdef %}
```

Load the Partial

```
from django.template.loader import get_template

partial = get_template("users.html#user_card")
html = partial.render({"user": user})
```

The returned template object contains the partial's content.

Partials are useful for reusable template fragments. They are a Django 6.0 feature, so do not assume this syntax is available in older Django versions.

---

14. HTML Autoescaping and Template Security

Django's DTL enables HTML autoescaping by default.

For example, a variable containing HTML-like text is generally escaped instead of being interpreted as executable HTML.

```
{{ user_input }}
```

This behavior helps reduce the risk of Cross-Site Scripting (XSS).

Security Best Practices

1. Keep autoescaping enabled for HTML templates.
2. Do not use `safe` with untrusted user input.
3. Never allow untrusted users to author templates that your application executes.
4. Use `{% csrf_token %}` in POST forms.
5. Validate and sanitize data appropriately before using it in sensitive contexts.
6. Use Django's URL reversing with `{% url %}` rather than hardcoding internal URLs.
7. Do not expose secret or sensitive information in the template context.

Important: Template autoescaping does not make every use of untrusted data safe in every context. JavaScript, CSS, URLs, and other contexts require appropriate handling.

---

15. Common Template Errors

Error 1: TemplateDoesNotExist

Cause: Django cannot find the requested template.

Solution:

- Check the template filename.
- Check the `DIRS` setting.
- Check whether the app is in `INSTALLED_APPS`.
- Check the app's `templates/` directory.
- Use the correct relative path.

Error 2: TemplateSyntaxError

Cause: Invalid template syntax.

Incorrect:

```
{% if user.is_authenticated %}
    <p>Welcome</p>
```

Correct:

```
{% if user.is_authenticated %}
    <p>Welcome</p>
{% endif %}
```

Error 3: Variable Displays Nothing

Possible causes:

- The variable was not passed in the context.
- The variable name is misspelled.
- The variable resolves to an empty or false value.
- A dictionary key or attribute does not exist.

Example:

```
return render(request, "home.html", {"username": "Omar"})
```

```
{{ username }}
```

Error 4: Static Files Do Not Load

Check:

- `{% load static %}` is present.
- `STATIC_URL` is configured.
- The file exists at the expected location.
- Your static-file setup is correct for the development or production environment.

Error 5: Template Logic Is Too Complex

Solution: Move database queries, business logic, and complex calculations into views, models, or other Python code. Keep templates focused on presentation.

---

16. Best Practices

- Use template inheritance with a shared `base.html`.
- Organize templates into application-specific directories.
- Use `{% url %}` for internal links.
- Use `{% load static %}` for static files.
- Use context processors only for genuinely shared data.
- Keep business logic out of templates.
- Keep HTML autoescaping enabled.
- Use clear, descriptive template names.

- Prefer Django's `render()` shortcut for ordinary view rendering.
- Use DTL unless another template engine provides a clear advantage.
- Test templates and verify that expected data is displayed.

---

17. Quick Revision

Topic	Syntax or API	Purpose
Variable	`{{ name }}`	Display a value
Tag	`{% if condition %}`	Perform template logic
Filter	`{{ name|upper }}`	Transform a value
Comment	`{# comment #}`	Add a single-line comment
Multi-line comment	`{% comment %}...{% endcomment %}`	Add a non-rendered block
Loop	`{% for item in items %}`	Iterate over items

Condition	`{% if condition %}`	Conditionally display content
Inheritance	`{% extends "base.html" %}`	Reuse a parent layout
Block	`{% block content %}`	Define a replaceable section
URL	`{% url 'profile' %}`	Reverse a named URL pattern
Static file	`{% static 'css/style.css' %}`	Generate a static-file URL
CSRF	`{% csrf_token %}`	Add a CSRF token

Render	`render(request, template, context)`	Return a rendered HTTP response
Load template	`get_template()`	Retrieve a template object
Select template	`select_template()`	Choose the first existing template
Render to string	`render_to_string()`	Return rendered HTML as a string
Configuration	`TEMPLATES`	Configure template engines

---

18. Practice Projects

Practice these tasks to strengthen your understanding:

1. Create a home page using template variables.
2. Display a list of students using a `for` loop.
3. Show different messages using `if`, `elif`, and `else`.
4. Format dates and prices using template filters.
5. Create `base.html` and extend it in three different pages.
6. Create a product list using model data passed through a view.
7. Configure a project-level templates directory.

8. Write a custom context processor.
9. Load a template with `get_template()` and render it.
10. Configure Jinja2 as an alternative template backend.
11. Build a reusable template partial if you are using Django 6.0.
12. Test template escaping with harmless HTML-like input and verify that it is not executed.

Conclusion

Django Templates separate the presentation layer from Python application logic. By learning variables, tags, filters, inheritance, contexts, template loading, configuration, and security, you can build maintainable and dynamic Django web applications.